Funciones definidas por el usuario (UDF's) – Common Tables Expressions (CTE)

Funciones

¿Qué es una Función en MySQL?

Una función en MySQL es un bloque de código SQL que recibe cero o más argumentos, realiza una serie de operaciones y devuelve un único valor. Al igual que los procedimientos almacenados, las funciones permiten encapsular lógica y reutilizar código. Se invoca con la sentencia SELECT o dentro de una expresión.

Tipos de Funciones en MySQL (las que vamos a estudiar)

Cualquier combinación de valores, operadores y funciones que se evalúan para producir un valor

1. **Escalares (entre ellas las UDF's):**
   - Devuelven un único valor y se pueden utilizar en cualquier lugar donde se permita una **expresión**.
   - Ejemplo: calcular_descuento (precio DECIMAL(7,2))

   Nota: Existe el caso especial de funciones escalares "tipo tabla" que devuelven un conjunto de registros en lugar de un único valor. Son menos comunes y no serán nuestro objeto de estudio.

2. **Agregadas (algunas que ya conocemos):**
   - Estas funciones realizan cálculos sobre un conjunto de valores y devuelven un valor único.
   - Ejemplo: SUM, AVG, MAX, MIN.

Funciones Escalares

Las funciones escalares operan sobre una sola fila de datos y devuelven un único valor. Son útiles cuando necesitamos realizar una operación sobre un valor individual en una consulta.

Ejemplos

1. **Funciones Matemáticas:**
   - ABS(x): Devuelve el valor absoluto de x.
   - ROUND(x, d): Redondea x a d decimales.
   - SQRT(x): Devuelve la raíz cuadrada de x.
2. **Funciones de Cadena:**
   - UPPER(str): Convierte una cadena a mayúsculas.
   - LOWER(str): Convierte una cadena a minúsculas.
   - CONCAT(str1, str2, ...): Concatenar varias cadenas.
3. **Funciones de Fecha:**
   - NOW(): Devuelve la fecha y hora actuales.
   - DATE_ADD(date, INTERVAL expr type): Añade un intervalo a una fecha.
   - YEAR(date): Devuelve el año de una fecha.

Funciones Agregadas

Las funciones agregadas operan sobre un conjunto de filas y devuelven un único valor que resume la información de ese conjunto. Son útiles cuando necesitamos calcular estadísticas o resúmenes de datos.

Ejemplos de Funciones Agregadas

- Max()
- Min()
- Sum()
- Avg()
- Count()

Tabla Comparativa entre estos tipos de funciones

Característica	Funciones Escalares	Funciones Agregadas
Operación	Sobre una sola fila	Sobre un conjunto de filas
Resultado	Un único valor	Un único valor que resume el conjunto
Uso	Cálculos, transformaciones y operaciones	Resúmenes, estadísticas y agregaciones
Ejemplos	ABS, ROUND, UPPER, NOW	SUM, AVG, COUNT, MAX, MIN
Aplicación	SELECT ABS(columna) FROM tabla	SELECT SUM(columna) FROM tabla

---

Sintaxis Básica de una Función

```
CREATE FUNCTION nombre_funcion(parametro1 tipo_dato, parametro2 tipo_dato, ...)
RETURNS tipo_dato_retorno
BEGIN
        /* aquí escribimos el codigo de la función */
RETURN valor_de_retorno;
END;
```

Ejemplo: Función para Calcular el Descuento (**UDF**)

```
CREATE FUNCTION calcular_descuento(precio DECIMAL(7,2))
DETERMINISTICS
RETURNS DECIMAL(7,2)
BEGIN
        DECLARE descuento DECIMAL(7,2);
        SET descuento = precio * 0.1;
        RETURN precio - descuento;
END;
```

Indica que la función devuelve el mismo resultado para los mismos parámetros de entrada.

Cuidado!!! (no es lo mismo)

Funciones Determinísticas

Una función se considera **determinística** si devuelve el **mismo resultado** cada vez que se invoca con los mismos parámetros (no necesariamente con los mismos valores). Esto significa que su salida es predecible y constante para los mismos valores de entrada. Ejemplos típicos son ABS() o LENGTH().

Ejemplos:

```
SELECT ABS(-5) AS Resultado1, ABS(-5) AS Resultado2;
```

```
CREATE FUNCTION CalcularImpuesto(precio DECIMAL(10, 2))
RETURNS DECIMAL(10, 2)
DETERMINISTIC
BEGIN
    RETURN precio * 0.21;
END;
```

Funciones No Determinísticas

Por otro lado, una función se considera **no determinística** si su salida puede variar, aunque se invoque con los mismos parámetros. Esto puede suceder si la función depende de variables externas, datos en tablas que pueden cambiar, o funciones como NOW(), que devuelven la fecha y hora actual.

Ejemplo:

```
SELECT NOW() AS Resultado1, NOW() AS Resultado2;
```

```
CREATE FUNCTION CalcularImpuestoVariable(precio DECIMAL(10, 2))
RETURNS DECIMAL(10, 2)
NOT DETERMINISTIC
BEGIN
    RETURN precio * (IF(DATE(NOW()) < '2025-01-01', 0.18, 0.21));
END;
```

Importancia del Determinismo:

- **Optimización**: Saber que una función es determinística permite al motor de base de datos optimizar consultas y reutilizar resultados.
- **Consistencia**: Garantiza resultados consistentes y predecibles para las mismas entradas.
- **Caché**: Funciones determinísticas pueden ser evaluadas una vez y almacenadas en caché para mejorar el rendimiento.
- **Replicación y Restauración**: Especificar el determinismo ayuda en la replicación de bases de datos. MySQL necesita saber si los resultados de las funciones son predecibles para mantener la consistencia entre el servidor maestro y los servidores esclavos.
- **Documentación y Mantenimiento**: Indicar si una función es determinística o no ayuda a otros desarrolladores a comprender mejor cómo funciona la función y qué esperar de ella, lo que facilita el mantenimiento del código.

Por otro lado...

Si no especificamos DETERMINISTIC o NOT DETERMINISTIC, MySQL asumirá por defecto que la función es **no determinística**. Esto puede llevar a menos optimizaciones y afectar el rendimiento en ciertos casos.

Por lo tanto, aunque no es obligatorio, es una buena práctica especificar si una función es determinística o no para aprovechar al máximo las capacidades de optimización y mantener la claridad del código.

En definitiva, para poder crear una función en MySQL es necesario indicar al menos una de estas tres características:

- **DETERMINISTIC (**estas son el objeto de nuestro estudio**)**
- NO SQL (Indica que la función no contiene sentencias SQL)
- READS SQL DATA (Indica que la función no modifica los datos de la base de datos y que contiene sentencias de lectura de datos)

Si no se indica al menos una de estas características obtendremos mensaje de error.

Comparación entre Funciones y Procedimientos Almacenados en MySQL

1. Funciones

• Retorno de un valor: El objetivo principal de una función es realizar un cálculo y devolver un único valor.

• Uso en expresiones: Pueden ser utilizadas directamente dentro de expresiones SQL, como en el SELECT, WHERE o HAVING.

• Sintaxis: Tienen una sintaxis específica que incluye la cláusula RETURNS para indicar el tipo de dato del valor a devolver.

• Limitaciones: Generalmente no realizan operaciones de modificación de datos (INSERT, UPDATE, DELETE) de forma directa.

• Ejemplos de uso:
  o Calcular el precio total de una venta.
  o Obtener el nombre completo de un cliente.
  o Verificar si un valor existe en una tabla.

2. Procedimientos Almacenados (SP)

• Conjunto de instrucciones: Son bloques de código SQL que pueden realizar múltiples operaciones, incluyendo consultas, actualizaciones y llamadas a otras funciones o procedimientos.

• No retornan un valor: Aunque pueden modificar datos, no devuelven un valor directamente. Para obtener resultados, se utilizan variables de salida o se generan conjuntos de resultados.

• Uso: Se invocan mediante la sentencia CALL.

• Flexibilidad: Ofrecen mayor flexibilidad al permitir realizar operaciones más complejas y controlar el flujo de ejecución del código.

• Ejemplos de uso:
  o Insertar nuevos registros en múltiples tablas.
  o Actualizar varios registros de una tabla.
  o Realizar transacciones complejas.

Tabla comparativa entre Funciones y SP

Característica	Funciones	Procedimientos Almacenados
Propósito principal	Calcular y devolver un valor	Realizar múltiples operaciones
Uso en expresiones	Sí	No
Retorno de valor	Sí, uno solo	No (en el sentido estricto), pero pueden modificar datos
Sintaxis	CREATE FUNCTION	CREATE PROCEDURE
Invocación	En expresiones SQL	CALL
Complejidad	Generalmente más simple	Pueden ser más complejos

Cuando utilizar una función

• Cálculos simples: Cuando necesitamos realizar operaciones matemáticas o manipulaciones de datos relativamente sencillas, una función es ideal. Por ejemplo, calcular el descuento de un producto, obtener la longitud de una cadena de texto o convertir una fecha a un formato específico.

• Validaciones: Las funciones son útiles para validar datos de entrada, como verificar sí un valor está dentro de un rango específico o si un formato es correcto.

• Reutilización en expresiones: Podemos utilizar funciones directamente en las consultas SQL, lo que facilita la reutilización de código y mejora la legibilidad.

• Retorno de un único valor: Si necesitamos obtener un solo resultado como salida, una función es la opción más directa.

Cuando utilizar un procedimiento almacenado

• Operaciones complejas: Cuando requerimos realizar múltiples operaciones, como insertar, actualizar o eliminar datos en varias tablas, un SP es más adecuado.

• Transacciones: Los SP son ideales para gestionar transacciones, asegurando que un conjunto de operaciones se ejecute como una unidad atómica.

• Procedimientos de negocio: Si tenemos una serie de pasos bien definidos que conforman un proceso de negocio, un SP puede encapsular toda esa lógica.

• Control de flujo: Los SP nos permiten utilizar estructuras de control como IF, ELSE, WHILE y FOR para tomar decisiones y repetir bloques de código.

• Parámetros de entrada y salida: Los SP pueden aceptar múltiples parámetros de entrada y devolver resultados a través de parámetros de salida o conjuntos de resultados.

Ejemplo:

Supongamos que tenemos una tienda en línea y necesitamos calcular el precio final de un pedido, aplicando descuentos y calculando impuestos.

• Función: Podemos a crear una función calcular_precio_final que tome como entrada el precio base, el descuento y el impuesto, y devuelva el precio final.

• Procedimiento almacenado: Podemos a crear un SP que calcule el precio final, actualice la tabla de pedidos y envíe un correo electrónico de confirmación.

Función para calcular el precio final de una venta

```
CREATE FUNCTION calcular_precio_final(venta_id INT, descuento DECIMAL(7,2),
impuesto DECIMAL(7,2))
RETURNS DECIMAL(10,2)
BEGIN
DECLARE subtotal DECIMAL(10,2);
DECLARE total DECIMAL(10,2);
        SELECT precio * cantidad INTO subtotal
        FROM ventas
        WHERE venta_id = venta_id;
        SET total = subtotal * (1 - descuento) * (1 + impuesto);
        RETURN total;
END;
```

Procedimiento almacenado para actualizar el precio final en la tabla ventas

```
CREATE PROCEDURE actualizar_precio_final(venta_id INT, descuento DECIMAL(7,2),
impuesto DECIMAL(7,2))
BEGIN
        UPDATE ventas
        SET precio = calcular_precio_final(venta_id, descuento, impuesto)
        WHERE venta_id = venta_id;
END;
```

Llamado al SP:

CALL actualizar_precio_final(10, 0.1, 0.21); -- Actualizamos la venta con ID 10, con un descuento del 10% y un impuesto del 21%.

Para tener en cuenta…

1. ¿Una función puede llamar a otra?

Sí, en MySQL, una función puede llamar a otra función dentro de su definición. Esto es útil para encapsular lógica y reutilizarla en múltiples lugares. Sin embargo, esto tiene que cumplir ciertas restricciones:

- La función que se llama debe ser **determinística** (en la mayoría de los casos).
- Si la función llamada contiene sentencias SQL, podría restringirse según el contexto.

```
-- Función para calcular el descuento
CREATE FUNCTION CalcularDescuento(precio DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN precio * 0.10; -- 10% de descuento
END;

-- Función que aplica el descuento y calcula el precio final
CREATE FUNCTION CalcularPrecioFinal(precio DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN precio - CalcularDescuento(precio);
END;
```

2. ¿Una función puede modificar datos?

No, en MySQL **las funciones definidas por el usuario (UDF) no pueden realizar operaciones DML** como INSERT, UPDATE, o DELETE. Esto es una restricción del diseño de MySQL para garantizar que las funciones sean rápidas, seguras y enfocadas en cálculos.

- **Razón:** Las funciones están diseñadas para ser usadas dentro de consultas (SELECT, WHERE, etc.) y no para manipular directamente datos en tablas.

Otros tipos de funciones MySQL

3. Funciones de Ventana o Analíticas
- **¿Qué hacen?** Permiten realizar cálculos sobre un subconjunto de filas dentro de un conjunto mayor, sin agruparlas como las funciones agregadas. (ver Cursores)
- **Características especiales:** Usan cláusulas como OVER() con particiones y ordenamientos.
- **Ejemplos:**
  - ROW_NUMBER() (asigna un número de fila dentro de una partición),
  - RANK(), DENSE_RANK() (asignan rangos),
  - SUM() con OVER() (cálculos acumulativos).

4. Funciones de JSON
- **¿Qué hacen?** Ayudan a trabajar con datos en formato JSON.
- **Ejemplos:**
  - JSON_EXTRACT(): Extrae un valor de un documento JSON.
  - JSON_ARRAYAGG(): Genera un array en formato JSON a partir de un conjunto de filas.
  - JSON_OBJECT(): Crea un objeto JSON.

5. Funciones de Información
- **¿Qué hacen?** Devuelven detalles sobre el sistema, la base de datos o la sesión actual.
- **Ejemplos:**
  - USER(): Devuelve el nombre del usuario conectado.
  - DATABASE(): Muestra la base de datos actual.
  - VERSION(): Devuelve la versión del servidor.

6. Funciones de Control de Flujo
- **¿Qué hacen?** Permiten manejar condiciones lógicas dentro de las consultas.
- **Ejemplos:**
  - IF(): Devuelve un valor según una condición.
  - CASE: Permite definir múltiples condiciones y devolver resultados distintos.
  - COALESCE(): Devuelve el primer valor no nulo.

7. Funciones de Seguridad
- **¿Qué hacen?** Permiten trabajar con información relacionada a permisos y cifrado.
- **Ejemplos:**
  - PASSWORD(): Devuelve una versión cifrada de una cadena (aunque está deprecada).
  - SHA2(): Genera un hash criptográfico.

8. Funciones de Espaciales (Geométricas)
- **¿Qué hacen?** Trabajan con datos geoespaciales (en columnas de tipo GEOMETRY).
- **Ejemplos:**
  - ST_DISTANCE(): Calcula la distancia entre dos objetos geométricos.
  - ST_CONTAINS(): Verifica si un objeto geométrico contiene a otro.
  - ST_AREA(): Calcula el área de un polígono.

9. Funciones de Replicación
- **¿Qué hacen?** Ayudan a gestionar la replicación entre servidores.
- **Ejemplos:**
  - MASTER_POS_WAIT(): Espera a que el servidor maestro alcance una posición específica en los registros.

Introducción a las CTE (Common Table Expressions)

Una CTE (Expresión de Tabla Común) es una forma temporal de definir una tabla derivada en una consulta SQL. También podemos decir que son una característica de SQL que permite crear una "tabla temporal" que puede ser referenciada dentro de una consulta.

A diferencia de las subconsultas, las CTE son más legibles y reutilizables. Su uso se recomienda especialmente cuando una consulta tiene múltiples partes complejas o cuando se necesita hacer referencia a un mismo conjunto de datos varias veces dentro de una consulta (CTE recursiva).

Tipos de CTE

1. **CTE Simples:**
   - Una CTE simple se define usando la cláusula WITH seguida por una consulta SQL.
   - La tabla temporal creada por la CTE solo está disponible durante la ejecución de la consulta principal que la sigue.

2. **CTE Recursivas:**
   - Las CTE recursivas permiten hacer referencia a sí mismas, facilitando el trabajo con datos jerárquicos o con estructuras recursivas, como árboles genealógicos, estructuras organizativas, o datos que requieren cálculos iterativos.

Sintaxis básica de una CTE (simple):

```
WITH mi_CTE AS
(
        SELECT columnas
        FROM tabla
        WHERE condiciones
)
```

Consulta que define la CTE

Consulta principal que utiliza la CTE

```
SELECT columnas
FROM mi_CTE;
```

Consulta principal (utiliza la CTE)

- WITH: Palabra clave que introduce la CTE.
- mi_CTE: Nombre temporal de la CTE.
- La consulta después de WITH define los datos que estarán disponibles dentro de la CTE.
- La consulta principal puede usar la CTE como si fuera una tabla temporal.

Sintaxis de una CTE recursiva (solo a modo de ejemplo):

```
WITH RECURSIVE nombre_cte_recursiva AS
(
   -- Este seria el "punto de partida" de la recursión
   SELECT columna1, columna2, ...
   FROM tabla
   WHERE condición_inicial

   UNION ALL

   -- Aquí podemos ver la recursividad (se hace referencia a la CTE)
   SELECT columna1, columna2, ...
   FROM tabla
   JOIN nombre_cte_recursiva ON condición_recursiva
)
```

Consulta principal que utiliza la CTE recursiva

```
SELECT columna1, columna2, ...
FROM nombre_cte_recursiva;
```

Ventajas de las CTE:

1. Legibilidad: Hace que las consultas sean más fáciles de leer y entender, especialmente cuando se tiene que trabajar con múltiples subconsultas.

2. Reutilización: Las CTE permiten usar el mismo conjunto de datos en diferentes partes de la consulta sin tener que escribir la subconsulta repetidamente.

3. Recursividad: Las CTE también pueden ser recursivas, lo que permite resolver problemas como el manejo de jerarquías o la iteración a través de conjuntos de datos.

Ejemplos de CTE

Ejemplo 1: CTE básica para calcular promedios

```
WITH EmpleadosSalarios AS
(
    SELECT empleado_id, salario
    FROM Empleados
    WHERE salario > 3000
)

SELECT AVG(salario) AS PromedioSalario
FROM EmpleadosSalarios;
```

la CTE EmpleadosSalarios selecciona los empleados con un salario superior a 3000

la consulta principal calcula el salario promedio de esos empleados

Ejemplo 2: CTE con JOIN

```
WITH EmpleadosDepartamentos AS
(
    SELECT e.empleado_id, e.nombre, d.nombre_departamento
    FROM Empleados e
    JOIN Departamentos d ON e.departamento_id = d.departamento_id
)

SELECT *
FROM EmpleadosDepartamentos
ORDER BY nombre_departamento;
```

Ejemplo 3: CTE recursiva

la CTE recursiva Jerarquía_empleados se utiliza para construir una jerarquía de empleados y sus jefes

```
WITH RECURSIVE jerarquia_empleados AS
(
   /*obtener el jefe inicial*/

   SELECT id, nombre, id_jefe
   FROM empleados
   WHERE id_jefe IS NULL  /*Comenzamos con el jefe máximo*/

   UNION ALL

   /*Parte recursiva: obtener los empleados supervisados por los empleados obtenidos en la
   parte inicial*/

   SELECT e.id, e.nombre, e.id_jefe
   FROM empleados e
   JOIN jerarquia_empleados je ON e.id_jefe = je.id
)
```

/*consulta ppal (llamado a la CTE recursiva)*/

```
SELECT *
FROM jerarquia_empleados;
```

Diferencias entre SubConsultas y CTE's (tabla comparativa)

A continuación, tenemos una tabla comparativa en la que podemos observar las características principales que comparten o no, las subquerys y las cte's.

Característica	Subconsultas	CTE (Common Table Expressions)
Definición	Consultas anidadas dentro de una consulta principal	Bloques de consulta temporales referenciables
Sintaxis	Parte de las cláusulas SELECT, FROM, WHERE, etc.	Definida con WITH y puede ser recursiva
Legibilidad	Puede volverse compleja y difícil de leer	Mejora la legibilidad dividiendo consultas complejas
Reutilización	No puede ser reutilizada dentro de la misma consulta	Puede ser referenciada múltiples veces en una consulta
Recursividad	No admite recursividad	Admite recursividad (útil para datos jerárquicos)
Compatibilidad	Compatibles con casi todas las versiones de SQL	Requiere versiones posteriores a MySQL 8.0

Ejemplo (para comparar)

1. *Subconsulta para Obtener el Cliente con más facturacion*

```
SELECT cliente_id, nombre
FROM clientes
WHERE cliente_id = (SELECT cliente_id
                    FROM ventas
                    GROUP BY cliente_id
                    ORDER BY SUM(total) DESC
                    LIMIT 1);
```

2. *CTE para Obtener el Cliente con más facturacion*

```
WITH ventas_totales AS
(
   SELECT cliente_id, SUM(total) AS total_ventas
   FROM ventas
   GROUP BY cliente_id
)

SELECT c.cliente_id, c.nombre
FROM clientes c
JOIN ventas_totales v ON c.cliente_id = v.cliente_id
ORDER BY v.total_ventas DESC
LIMIT 1;
```

Entonces, por lo visto hasta aquí, ¿cuándo nos conviene usar una SubQuery y cuando una CTE?

CTE cuando:

- Queremos mejorar la legibilidad y estructura del código.
- Necesitamos reutilizar resultados intermedios.
- Trabajamos con datos jerárquicos o recursivos.
- Deseamos separar la lógica de negocio en componentes claros.
- Utilizamos versiones modernas de MySQL (8.0 o superior).

Subconsultas cuando:

- La consulta sea simple y no necesite ser reutilizada en múltiples lugares.
- No tengamos que trabajar con datos recursivos.
- La compatibilidad con versiones más antiguas de MySQL sea un factor.

Resumen General (comparativo)

Elemento	Relación con UDF	¿Qué puede hacer?	¿Qué no puede hacer?
Funciones (UDF)	Es autónoma, pero **puede ser invocada en SP, triggers, vistas, consultas y CTE**.	- Devuelve un valor (o tabla si es una UDF tipo tabla). - Se usa en consultas SQL (SELECT, WHERE, etc.) para cálculos o transformaciones.	- No realiza operaciones DML (INSERT, UPDATE, DELETE). - No gestiona flujo transaccional ni errores (no usa handlers).
Procedimientos (SP)	Puede **invocar UDF** para cálculos específicos.	- Ejecuta tareas complejas, incluyendo operaciones DML. - Usa transacciones, cursores y lógica avanzada.	- No se puede llamar directamente desde SELECT. - No suele devolver valores directamente como las UDF.
Triggers	Puede **usar UDF** para lógica de cálculos y SP para procesos más complejos.	- Responde automáticamente a eventos DML (INSERT, UPDATE, DELETE). - Permite automatizar lógica asociada a cambios en tablas.	- No se invoca directamente (se activa solo por eventos). - No incluye cálculos complejos sin UDF/ayuda externa.
Handlers	No tienen relación directa con UDF. Exclusivos de SP y triggers para manejar errores y condiciones especiales.	- Permiten manejar errores como NOT FOUND o interrumpir el flujo de un proceso SQL. - Se usan en combinación con cursores y transacciones.	- No aplicables en UDF, vistas o CTE, ya que estos no gestionan flujo ni errores transaccionales.
Vistas	Las UDF **pueden ser usadas dentro de vistas** para cálculos o transformaciones reutilizables.	- Ofrecen una visión estructurada basada en consultas complejas. - Se utilizan para simplificar el acceso a datos y abstracción lógica.	- No admiten parámetros (como las UDF o SP). - No realizan operaciones DML directamente (son solo para consultas).
CTE (Common Table Expressions)	Una UDF **puede ser invocada dentro de una CTE** para cálculos específicos.	- Facilitan la creación de subconsultas temporales reutilizables en una consulta. - Simplifican el trabajo con jerarquías o cálculos complejos.	- No almacenan datos; solo son "temporales" dentro de la consulta donde se declaran. - No se invocan de manera independiente.
Vistas Materializadas	No están relacionadas directamente con UDF, pero **pueden depender de lógica previa calculada con estas**.	- Permiten almacenar los resultados de una consulta. - Mejoran el rendimiento en operaciones de consulta frecuente sobre datos calculados.	- Requieren procesos de actualización manual o programada. - No permiten actualizaciones directas de su contenido.